Software Test Plan (STP)

Digital Library and Booking System (DLBS)

Project: Digital Library and Booking System (DLBS)

Version: 1.0

Authors: Marreddy Rushieswar Reddy (QA Lead), with Mohammed Sinan M T, Mallela Rishyendra and Nagalapuram Sai Mukesh Reddy

Date: 30-10-2026

Status: Submitted (baseline before Sprint 1)

Revision history

Version	Date	Author	Change summary
1.0	30-10-2026	Team DLBS (QA Lead)	Baseline test plan for Sprint 1 and Sprint 2 with the full test case catalogue

Table of Contents

1. Introduction 3
2. Test items 3
3. Features to be tested 3
4. Features not to be tested 4
5. Test approach / strategy 4
5.1 Security validation 4
6. Test environment 4
7. Test schedule 5
8. Test deliverables 5
9. Roles and responsibilities 5
10. Risks and mitigation 6
11. Assumptions and dependencies 6
12. Suspension and resumption criteria 6
13. Test case management and traceability 6
14. Test metrics and reporting 9
15. Approvals 9

1. Introduction

Purpose: this document defines how the Digital Library and Booking System v1.0 is tested: objectives, scope, strategy, environment, schedule, responsibilities and the test cases that verify each requirement.

Scope: all four modules (Book Catalog, Users and Reading Tracking, Booking and Borrowing, Recommendation Engine), the web UI, the REST API, the CI/CD pipeline and the deployed application. Payment, notifications and third-party platform internals are excluded (section 4).

References: SRS v1.0 (requirements and RTM), SAD v1.0 (architecture and design), docs/api-contract.md, the member plans (v2) and the course announcement for sprint dates.

Table 1. Definitions.

Term	Meaning
STP / SRS / SAD	Software Test Plan / Requirements Specification / Architecture and Design
TC	Test case, numbered TC-<area><number>: C catalog, U users, B booking, R recommendations, P performance, S security, Q quality and pipeline, UX usability
RTM	Requirements Traceability Matrix (SRS section 8; live copy in docs/qa/rtm.md)
Smoke test	A short end-to-end check that the main flow works after a merge or deployment
Regression	Re-running the full suite to make sure nothing that worked has broken
Coverage	Share of code lines run by the unit tests, measured with pytest-cov

2. Test items

Item	Location	Owner
Book Catalog module	src/catalog	Mallela Rishyendra
Users and Reading Tracking module	src/users	Nagalapuram Sai Mukesh Reddy
Booking and Borrowing module	src/booking	Marreddy Rushieswar Reddy
Recommendation Engine	src/recommender, data/	Mohammed Sinan M T
REST API application and /health	src/main.py	Nagalapuram Sai Mukesh Reddy
Web UI pages	app shell and module screens	Mallela Rishyendra with each owner
CI/CD pipeline and Docker setup	.github/workflows, Dockerfile, docker-compose.yml	Nagalapuram Sai Mukesh Reddy
Seed scripts	scripts/seed.py	Marreddy Rushieswar Reddy

3. Features to be tested

Features are mapped to the SRS requirement IDs. Every requirement has at least one test case.

Feature	SRS requirements	Test cases
Book catalog	FR-C1–FR-C9, NFR-C1–NFR-C3	TC-C01–TC-C12, TC-P01, TC-Q01
Accounts and reading tracking	FR-U1–FR-U8, NFR-U1–NFR-U4	TC-U01–TC-U13, TC-S01–TC-S03, TC-Q01
Booking and borrowing	FR-B1–FR-B10, NFR-B1–NFR-B3	TC-B01–TC-B12, TC-P02, TC-Q01
Recommendations	FR-R1–FR-R6, NFR-R1–NFR-R3	TC-R01–TC-R08, TC-P03, TC-Q01
Security	SR-01–SR-07	TC-S01–TC-S08, TC-C09, TC-C12
CI/CD and deployment	NFR-G1	TC-Q02

Feature	SRS requirements	Test cases
Usability and accessibility	NFR-G2	TC-UX01

4. Features not to be tested

- Payments, fines, e-mail or SMS notifications and reading e-book files inside the app: out of scope in the SRS.
- Internals of the hosting platform, GitHub Actions and browser engines: owned by their vendors.
- Accuracy of the Goodbooks-10k source data beyond the schema and the derived fields we generate.
- Load beyond the NFR targets (10,000 books and 1,000 users).
- Item-based collaborative filtering, unless the stretch story is delivered.

5. Test approach / strategy

Level	Who	What	Tools	When
Unit	Each module owner	Functions, validation, state rules, scoring	pytest, pytest-cov	With every story
Integration	QA Lead	API endpoints and calls between modules	pytest + TestClient	As features merge; 4–6 Nov across modules
System	QA Lead	End-to-end flows on Docker and the deployed app: log in → search → reserve → borrow → read → recommendation	Browser, Swagger UI, scripts	Smoke 3 Nov; full 7 Nov
Acceptance	Whole team	SRS acceptance criteria shown in the demo	Demo checklist	Sprint reviews and from 9 Nov

Test types: functional, regression (full suite before each sprint review and at code freeze), performance (TC-P01–TC-P03), concurrency (TC-B11), security (TC-S01–TC-S08) and usability and accessibility (TC-UX01).

Automation: unit and integration tests are automated with pytest. From Sprint 1 the pipeline builds and lints every push and pull request. In Sprint 2 the test stage runs every test with a coverage gate of 50% from 4 Nov, rising to 70% from 6 Nov. UI checks and the accessibility audit are manual.

Entry criteria: the story is merged to main through a reviewed pull request, the pipeline is green, and the test data and environment are ready.

Exit criteria: 100% of planned test cases executed; every critical and high defect fixed and retested; every module at or above 70% coverage; every RTM row has at least one passed test; the pipeline is green on main; and the deployed app passes the smoke test.

5.1 Security validation

- Password storage: bcrypt hashes only, never plain text in the database or logs (TC-S01).
- Tokens: missing, tampered and expired tokens are rejected (TC-S02, TC-S03).
- Authorisation: role checks on admin endpoints and ownership checks on members' records (TC-S05, TC-S06, TC-C09).
- Input handling: injection strings and invalid input are rejected or treated as text (TC-S04, TC-C12).
- Secrets and transport: no secrets in the repository; HTTPS on the deployed app (TC-S07, TC-S08).

6. Test environment

Environment	Set-up	Used for
Local	Developer laptop, Python 3.11, SQLite, pytest	Unit and integration tests while coding

Environment	Set-up	Used for
CI	GitHub Actions ubuntu-latest, Python 3.11, SQLite in memory	Build, lint, all automated tests and coverage on every push and pull request
Docker	docker-compose with the app and PostgreSQL	System and performance tests close to production
Deployed	Free cloud host, PostgreSQL, HTTPS	Smoke test after deploy, regression, TC-S08, acceptance demo

Tools: pytest, pytest-cov, FastAPI TestClient, Swagger UI at /docs, browser developer tools and Lighthouse, Python timing (time.perf_counter) and threads for the performance and concurrency tests, Jira for defects, GitHub Actions for automated runs.

Test data: data/books_sample.csv (about 20 books) for unit tests; data/books.csv (10,000 books from Goodbooks-10k) for performance tests; scripts/seed.py for demo members, one admin and synthetic reading records; accounts created by test fixtures. No real personal data is used.

7. Test schedule

Dates follow the course announcement: Sprint 1 runs 30 Oct – 3 Nov 2026, Sprint 2 runs 3 Nov – 8 Nov 2026, and presentations start on 9 Nov 2026.

Date	Milestone
30 Oct	STP baselined and submitted with the SRS and SAD; Sprint 1 starts; pipeline v1 (build, lint) on every push
30 Oct – 3 Nov	Unit tests written with each Sprint 1 story; QA Lead writes the Sprint 1 test cases
3 Nov	Sprint 1 review; first cross-module smoke test on main (log in, search, reserve)
4 Nov	Test stage switched on in CI with a 50% coverage gate
4 – 6 Nov	Integration tests across modules; performance and concurrency tests; defects logged and retested
6 Nov	Coverage gate raised to 70%
7 Nov	Code freeze; full regression on main and on the deployed app; security checks
8 Nov	Test summary report; Sprint 2 review and retrospective
From 9 Nov	Acceptance demo in the presentation

8. Test deliverables

- Test plan (this document), with the test case catalogue in section 13.
- Test cases and the live RTM: docs/qa/test-cases.md and docs/qa/rtm.md.
- Automated test scripts: tests/ in the repository.
- Test data: data/books_sample.csv, data/books.csv and scripts/seed.py.
- Execution logs and coverage reports: GitHub Actions runs.
- Defect reports: Jira Bug cards.
- Test summary report: docs/qa/test-summary.md (8 Nov).

9. Roles and responsibilities

Role	Name	Responsibility
QA Lead	Marreddy Rushieswar Reddy	Owns this plan; writes integration and system test cases; keeps the RTM; triages and retests defects; writes the test summary report
Test engineer / developer (Recommendation)	Mohammed Sinan M T	Unit tests for the recommender; Precision@5 and Recall@5 evaluation; TC-P03

Role	Name	Responsibility
Test engineer / developer (Catalog) and Scrum Master	Mallela Rishyendra	Unit tests for the catalog; TC-P01; keeps Bug cards moving on the Jira board
Test engineer / developer (Users and Reading) and DevOps	Nagalapuram Sai Mukesh Reddy	Unit tests for accounts and reading; CI test stage and coverage gate; TC-Q02, TC-S07, TC-S08
Product Owner (acting)	Mallela Rishyendra for the team; the course TA as customer	Approves test results and release readiness at each sprint review

10. Risks and mitigation

Risk	Mitigation
Late integration between modules	Stubs and the API contract from 29 Oct; smoke test on 3 Nov
Tests interfering through the shared database	Each test uses a fresh in-memory SQLite database from a fixture
Missing test data	books_sample.csv and fixtures; seed script shared early
Flaky timing tests	Fixed dataset, a warm-up call, 20 runs and thresholds with some margin
Test environment down	docker-compose locally as a backup environment
Compressed schedule (Sprint 2 ends 8 Nov)	High-priority requirements first; regression on 7 Nov

11. Assumptions and dependencies

- Each module exposes the interfaces agreed in docs/api-contract.md.
- The seed script and both book files are ready by 3 Nov.
- A free host with PostgreSQL and HTTPS is available for the deployed tests.
- Developers fix and hand back critical and high defects within one working day.

12. Suspension and resumption criteria

Suspend testing if: the pipeline on main is red for more than 4 hours; the test environment is unavailable for more than 4 hours; the build blocks more than 30% of test cases; or a critical security defect is found.

Resume when: the fix is merged and the pipeline is green, the environment is restored, and the smoke test passes.

13. Test case management and traceability

Every test case records an ID, the linked requirement, preconditions, steps, the expected result, the actual result and pass or fail. The SRS RTM maps every requirement to these IDs; the QA Lead updates the status column after each run.

Defects are logged as Jira Bug cards with a title, steps to reproduce, expected and actual results, severity, the affected requirement and an owner, and move To Do → In Progress → In Review (retest by the QA Lead) → Done.

Severity	Meaning
Critical	Crash, data loss, security hole, or a core flow blocked
High	A High-priority requirement fails with no workaround
Medium	A requirement fails but a workaround exists
Low	Cosmetic or wording issue

Table 2. Test case catalogue (59 test cases).

ID	Requirement	Level	Preconditions	Steps	Expected result
TC-C01	FR-C1	Integration	Logged in as admin.	POST /api/books with every field and 3 copies; then search by its title.	201; available_copies = 3; the book appears in search.
TC-C02	FR-C2	Integration	Admin; book with 3 copies.	PUT a new title and total_copies = 5.	Title saved; available copies rise by 2.
TC-C03	FR-C2	Unit	Book with 2 copies, both on loan.	PUT total_copies = 1.	409; copy counts unchanged.
TC-C04	FR-C3	Integration	Book with one open loan.	DELETE the book; search; try to reserve; return the open loan.	Hidden from search and recommendations; reserve rejected; return succeeds.
TC-C05	FR-C4	Unit	Seeded catalog.	Search by part of a title, by author, by ISBN and by an unknown word.	Matching books for the first three; empty list for the last.
TC-C06	FR-C5	Unit	Books of several genres, some with 0 copies.	GET /api/books?genre=Fantasy&available=true.	Only Fantasy books with available copies.
TC-C07	FR-C6	Integration	Seeded catalog.	GET /api/books/{id}; GET an unknown id.	All fields including available copies; 404.
TC-C08	FR-C7	Unit	25 books.	GET page=2&size=10.	Items 11–20; total = 25.
TC-C09	FR-C8, SR-03	Integration	Member token.	POST, PUT and DELETE /api/books as a member and with no token.	403 and 401; nothing changes.
TC-C10	FR-C9	System	Seeded member with reading history.	Open the home page.	Up to 10 book cards, each with its reason text.
TC-C11	NFR-C2	Unit	A book with ISBN X exists.	Add another book with ISBN X.	409.
TC-C12	NFR-C2, SR-05	Unit	Admin.	Add a book without a title; with ISBN "12AB".	422 with the field errors.
TC-U01	FR-U1	Integration	None.	POST /api/auth/register with valid data.	201; role member; no password field in the response.
TC-U02	FR-U1	Unit	An account exists.	Register the same email again in another case.	409.
TC-U03	FR-U1, NFR-U3	Unit	None.	Register with email "abc", with password "short" and with password "password".	422 for each.
TC-U04	FR-U2	Integration	Registered member.	POST /api/auth/login with correct details.	200 with access_token and expires_in = 3600.
TC-U05	FR-U2	Unit	Registered member.	Log in with a wrong password; with an unknown email.	401 with the same message both times.
TC-U06	FR-U2	System	Logged in.	Log out in the UI, then open My Reading.	Token removed; redirected to the login page.
TC-U07	FR-U3	Integration	ADMIN_EMAIL and ADMIN_PASSWORD set.	Start the app; log in as admin; as a member call an admin endpoint.	Admin account exists with role admin; member gets 403.
TC-U08	FR-U4	Integration	Logged in.	GET /api/users/me; PUT a new name; try to set role.	Profile shown; name changed; role unchanged.
TC-U09	FR-U5	Unit	Logged in; book exists.	Set status reading; then status "paused".	Saved; then 422.
TC-U10	FR-U6	Unit	Book with 320 pages.	Record page 120; then page 320.	37.5%; then 100% and status finished.
TC-U11	FR-U6	Unit	Book with 320 pages.	Record 120%; -5; page 400.	422 each time; record unchanged.
TC-U12	FR-U7	Integration	Member with 3 records; a new member.	GET /api/reading/history for both; call reading.get_history(user_id).	Newest first; [] for the new member; same data from the function.
TC-U13	FR-U8	Unit	2 finished books; 1 at page 50.	GET /api/reading/stats.	books_finished = 2; pages_read = sum of current pages.
TC-B01	FR-B1, FR-B5	Integration	Book with 1 copy.	Member reserves it.	201 reserved; available copies 0.
TC-B02	FR-B9	Integration	After TC-B01.	A second member reserves the same book.	409; nothing changes.
TC-B03	FR-B3, FR-B5	Integration	Borrowed booking.	Return it.	Status returned; available copies +1.
TC-B04	FR-B8	Unit	Member with 5 active bookings.	Reserve a sixth book.	409.

ID	Requirement	Level	Preconditions	Steps	Expected result
TC-B05	FR-B10, FR-B5	Unit	Reservation made.	Move the clock 2 days and 1 minute ahead; read it.	Status expired; available copies restored.
TC-B06	FR-B1, FR-C3	Unit	Removed book.	Try to reserve it.	Rejected; nothing changes.
TC-B07	FR-B2	Unit	Reserved booking.	Borrow it.	Status borrowed; due date +14 days; available copies unchanged.
TC-B08	FR-B4, FR-B5	Unit	One reserved and one borrowed booking.	Cancel both.	Reserved one cancelled with copies +1; borrowed one rejected.
TC-B09	FR-B6	Unit	Borrowed booking.	Move the clock past the due date; GET /api/bookings/me.	overdue = true.
TC-B10	FR-B7	Integration	Two members with bookings.	Member A calls GET /api/bookings/me.	Only A's active and past bookings, with dates.
TC-B11	NFR-B1	Integration	Book with 1 copy.	Two threads reserve it at the same moment; repeat 20 times.	Exactly one succeeds each time; copies never below 0.
TC-B12	FR-B3	Unit	Reserved and cancelled bookings.	Return the reserved one; borrow the cancelled one.	Both rejected; states unchanged.
TC-R01	FR-R1	Integration	Seeded member.	GET /api/recommendations/me?limit=10.	At most 10 items, sorted by score, with every required field.
TC-R02	FR-R2	Unit	Member who finished 4 Fantasy books.	Get recommendations.	At least 3 of the top 5 are Fantasy.
TC-R03	FR-R3	Unit	New member.	Get recommendations.	Popular books; list not empty.
TC-R04	FR-R4	Unit	Member finished book A; book B removed.	Get recommendations.	Neither A nor B appears.
TC-R05	FR-R5	Unit	Seeded member.	Get recommendations.	Every item has a reason.
TC-R06	FR-R6	Integration	Admin and member tokens.	POST /api/recommendations/refresh as each.	Admin: success and cache cleared; member: 403.
TC-R07	NFR-R2	Unit	Stub class implementing Recommender.	Plug it in and call the endpoint.	Endpoint returns the stub's results; no other code changes.
TC-R08	FR-R1	Integration	None.	GET /api/recommendations/me with no token.	401.
TC-P01	NFR-C1	System	10,000 books seeded.	Time 20 searches.	95th percentile under 1 s.
TC-P02	NFR-B2	System	Seeded data.	Time 20 reserve, borrow and return calls.	Each under 1 s.
TC-P03	NFR-R1	System	10,000 books, 1,000 users.	Time 20 uncached recommendation calls.	Each under 2 s.
TC-S01	NFR-U1, SR-01	Unit	None.	Register; inspect the users table and the logs.	Hash starts with $2b$12$; the password appears nowhere.
TC-S02	NFR-U2, SR-02	Integration	None.	Call every protected endpoint with no token.	401.
TC-S03	SR-02	Unit	Valid token.	Call with an altered signature; with an expired token.	401 both times.
TC-S04	SR-05	Integration	Seeded data.	Search for ' OR 1=1 --; log in with email admin'--.	Treated as text; no error; no extra data.
TC-S05	SR-03	Integration	Member token.	Call every admin endpoint.	403 for each.
TC-S06	SR-04	Integration	Members A and B with records.	A reads and changes B's booking and reading record by id.	403 or 404; nothing changes.
TC-S07	SR-06	System	Repository on GitHub.	Scan the history for secrets; check .gitignore and the CI secrets.	No secrets found; .env ignored; CI uses GitHub Secrets.
TC-S08	SR-07	System	App deployed.	Open the http:// address.	Redirected to https:// with a valid certificate.
TC-Q01	NFR-C3, NFR-U4, NFR-B3, NFR-R3	System	All modules merged.	Run pytest with coverage in CI.	Every module at or above 70% (gate 50% from 4 Nov, 70% from 6 Nov).
TC-Q02	NFR-G1	System	Pipeline configured.	Open a pull request; merge it to main.	Build, lint and tests run; the merge deploys; /health returns 200.
TC-UX01	NFR-G2	System	App running.	Keyboard-only walk-through of log in, search and reserve; Lighthouse audit.	Every action reachable; every field labelled; score at least 90.

14. Test metrics and reporting

Metrics: percentage of test cases executed; pass and fail counts per module; defect density per module; defect ageing (days open); requirement coverage (RTM rows with a passed test); code coverage per module.

Reports: a CI result on every pull request; daily status at the stand-up (docs/standup-log.md); test results at the sprint reviews on 3 Nov and 8 Nov; and the final test summary report on 8 Nov.

15. Approvals

Role	Name	Signature / Date
QA Lead	Marreddy Rushieswar Reddy	
Dev Lead (architecture owner)	Mohammed Sinan M T	
Product Owner (acting)	Mallela Rishyendra	
Course Coordinator		